SE3002 - Software Quality Engineering

Assignment 2: Testing Report

Student Course Registration System: black-box and white-box testing with GitHub

Name	Roll No.	GitHub account
Junaid Javed	22P-9163	JuniJaved01
Mian Ayan	22P-9149	MianAyaan0316

Repository: https://github.com/JuniJaved01/SQE-Assignment-2

Item	Result
Test design	19 ECP + 26 BVA + 12 decision-table rows = 57 design rows. 12 exact duplicates were merged.
Test suites	56 black-box tests (tests/test_blackbox.py) + 10 white-box tests (tests/test_whitebox.py) = 66 tests
First run on the original code	Black-box only: 16 failed, 40 passed. With the white-box tests added: 19 failed, 47 passed.
Defects found	5 different defects (GitHub Issues #2 to #6). All fixed and closed.
Final run (after fixes, main)	66 tests passed. Statement coverage 100%, branch coverage 100%.
Cyclomatic complexity of registration_decision()	V(G) = 10 (E - N + 2 = 27 - 19 + 2, same as decision nodes + 1)
Pull requests	PR #1 (black-box), PR #7 (white-box), PR #8 (defect fixes). All merged.

1. Black-box test design and optimization

We used the requirements file (REQUIREMENTS.md, FR-01 to FR-15) as the only source for expected results. The black-box tests were designed before registration.py was opened, and we did not change any expected value to make a test pass. Every case starts from the same baseline: a student with 0 credits, 0 dues, no completed or registered courses, and course CS201 (3 credits, capacity 30, 0 enrolled, no prerequisites). Each case only lists what it changes. The full case sheets are in docs/blackbox_test_design.xlsx and docs/decision_table.xlsx.

1.1 Equivalence class partitioning (19 cases: ECP-01 to ECP-19)

Input	Requirement	Classes and test IDs
Outstanding dues	FR-04, FR-15	dues > 0 blocks (ECP-01); dues = 0 is valid (ECP-02); dues < 0 is invalid input (ECP-03)
Already registered	FR-05	student is already in the course (ECP-04)
Prerequisites	FR-03, FR-06	none required (ECP-05, ECP-16); all completed (ECP-06, ECP-17); some missing (ECP-07, ECP-18); none completed (ECP-08)
Credit hours	FR-02, FR-07	within 18 (ECP-09); over 18 (ECP-10); invalid course credits (ECP-13); invalid current credits (ECP-14); both invalid, to check the order of checks (ECP-19)
Availability	FR-08, FR-15	seats available (ECP-11); course full (ECP-12); invalid capacity (ECP-15)

1.2 Boundary value analysis (26 cases: BVA-01 to BVA-26)

Variable	Requirement	Boundary cases (test IDs)
Combined credits (max 18)	FR-01, FR-07	exactly 18 allowed (BVA-03, 05, 07, 23); 19 rejected (BVA-04, 06, 08, 24, 25); smallest totals (BVA-01, 02, 26)
Current credits (0 to 18)	FR-02	-1 and 19 are just outside (BVA-09, BVA-10)
Course credits (1 to 4)	FR-02	0 and 5 are just outside (BVA-11, BVA-12)
Dues near zero	FR-04, FR-15	1 blocks (BVA-13); 0 allowed (BVA-14); -1 invalid (BVA-15)
Enrolled vs capacity	FR-08, FR-15	29/30 one seat left (BVA-16); 30/30 full (BVA-17); 31/30 and -1 invalid (BVA-18, BVA-19); capacity 1 empty and full (BVA-20, 21); capacity 0 invalid (BVA-22)

1.3 Decision table testing

FR-09 gives the order of the checks: dues, already registered, prerequisites, credit limit, availability. Because a higher-priority check decides the result first, the 32 possible combinations reduce to 6 rules ('-' means the value does not matter).

Condition	R1	R2	R3	R4	R5	R6
Dues > 0	Y	N	N	N	N	N
Already registered	-	Y	N	N	N	N
Prerequisites missing	-	-	Y	N	N	N
Credits would exceed 18	-	-	-	Y	N	N
Course full	-	-	-	-	Y	N
Outcome	DUES_BLOCK	ALREADY_REGISTERED	PREREQUISITES_MISSING	CREDIT_LIMIT	WAITLISTED	REGISTERED
Test ID	DT-01	DT-02	DT-03	DT-04	DT-05	DT-06

Priority combinations (several conditions true at the same time, the highest priority must win): DT-07 (all five true) gives DUES_BLOCK; DT-08 (registered, prerequisites, credits, full) gives ALREADY_REGISTERED; DT-09 (prerequisites, credits, full) gives PREREQUISITES_MISSING; DT-10 (credits and full) gives CREDIT_LIMIT; DT-11 (dues and full) gives DUES_BLOCK; DT-12 (negative dues and capacity 0) raises 'Invalid dues', because dues are checked first (FR-15).

1.4 Implementation and optimization

The tests are in tests/test_blackbox.py and use assert and pytest.raises(). They are parametrized, so every test that runs carries the IDs of all the technique rows it covers (for example ECP-02|ECP-05|BVA-14|DT-06). This keeps the link from each technique to its test. There are also ten state tests (ST01 to ST10). They check that a successful registration updates the student and the course, that a waitlisted student is added only once, that a failed registration changes nothing, that registration_decision() does not change any state, and that withdrawal changes the right fields.

Optimization. We only treated two tests as duplicates when the function, the input and the expected output were all the same. Getting the same output alone is not enough. We found 12 exact duplicates and merged them, so the 57 design rows run as 45 tests (plus 11 state tests, which gives 56 black-box tests).

Executed once as	Rows merged	Same input
ECP-02|ECP-05|BVA-14|DT-06	4 rows (3 removed)	baseline student, expect REGISTERED
ECP-12|BVA-17|DT-05	3 rows (2 removed)	enrolled 30 of 30, expect WAITLISTED
ECP-01|DT-01	2 rows	dues = 500
ECP-04|DT-02	2 rows	already registered

Executed once as	Rows merged	Same input
ECP-07|DT-03	2 rows	completed {CS101}, required {CS101, MATH101}
ECP-10|DT-04	2 rows	current 16 + course 3 credits
ECP-13|BVA-12	2 rows	check_credit_limit(10, 5)
ECP-14|BVA-10	2 rows	check_credit_limit(19, 3)
ECP-15|BVA-22	2 rows	capacity 0

Kept as separate tests: ECP-01 (dues 500) and BVA-13 (dues 1) both give DUES_BLOCK, but one is a normal value and the other is the boundary. ECP-03 (-100) and BVA-15 (-1) are also different inputs.

1.5 First run before any fix

The first GitHub Actions run (PR #1, branch blackbox-testing) ran the suite on the original registration.py and ended with **16 failed, 40 passed**. We kept the failed runs as evidence (runs #1 to #12 on the blackbox-testing branch are all red). The failures come from five defects (section 4).

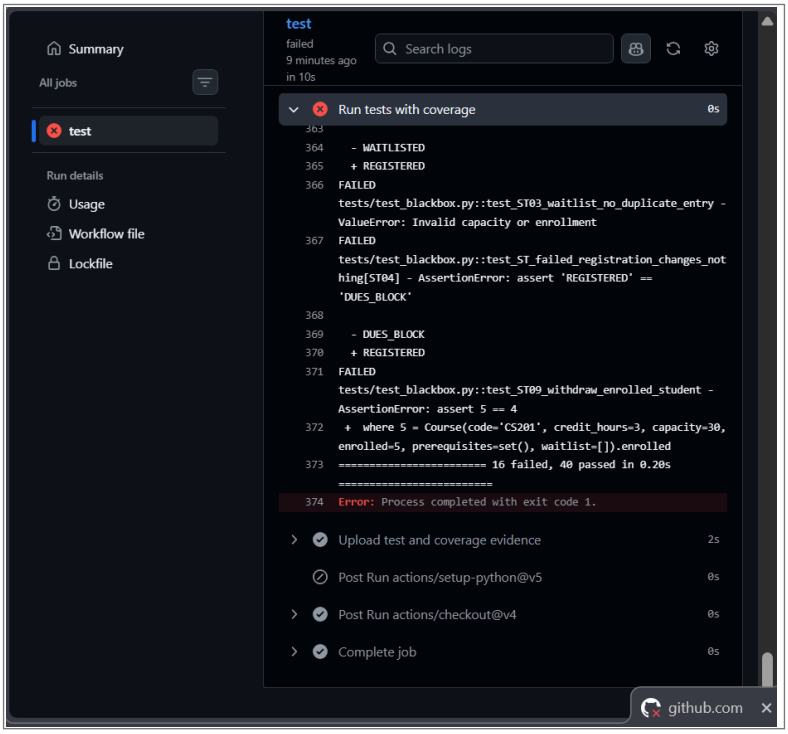


Figure 1. GitHub Actions log of the first run: 16 failed, 40 passed (exit code 1).

2. White-box analysis

2.1 Control flow graph and cyclomatic complexity

The control flow graph (CFG) of registration_decision() is in Figure 2 (also docs/cfg.png). **Convention for short-circuit conditions:** each atomic condition inside a compound condition is counted as its own decision node. Line 48 is *not (0 <= enrolled <= capacity) or capacity < 1*, so it has three atomic conditions: A = 0 <= enrolled, B = enrolled <= capacity, C = capacity < 1. The calls to prerequisites_satisfied() and check_credit_limit() count as one decision each, because their insides have their own CFGs.

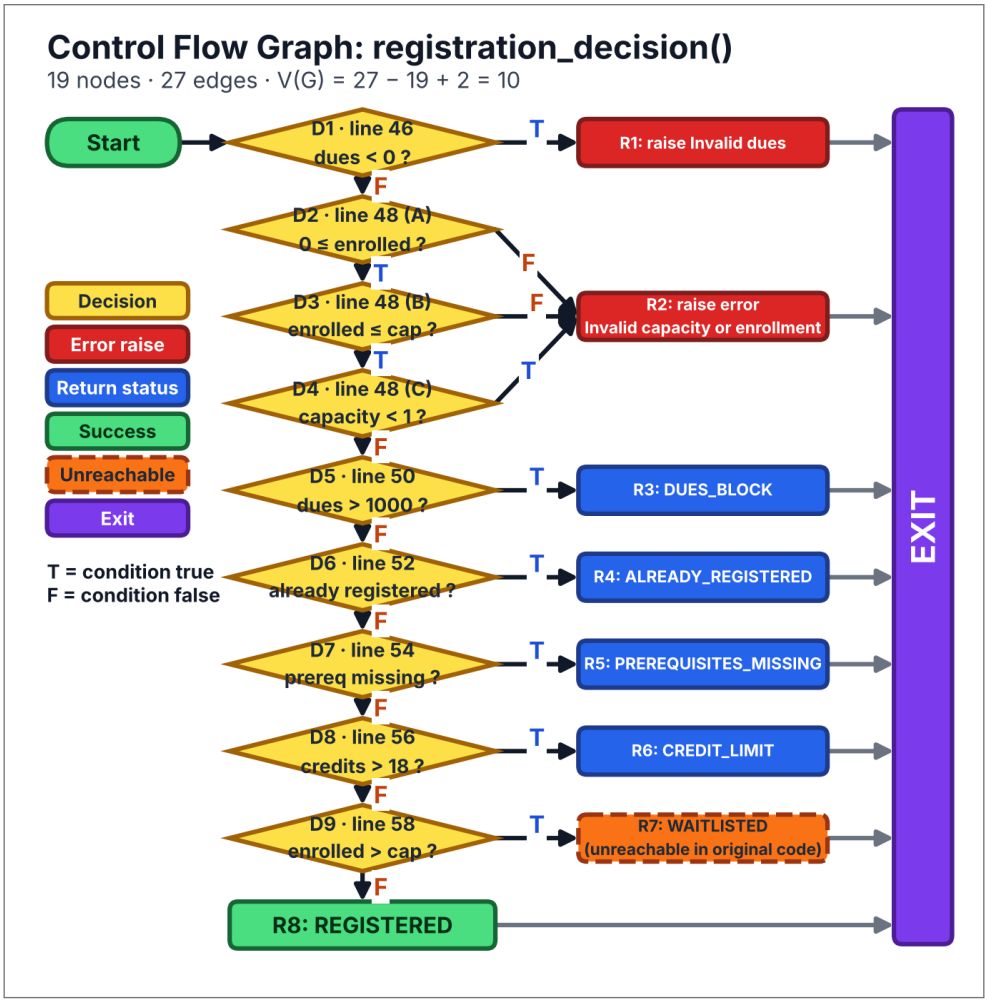


Figure 2. Control flow graph of registration_decision(): 9 decision nodes, 8 return/raise nodes, Start and Exit.

Measure	Value
Nodes N	19 (Start, 9 decisions D1 to D9, 8 return/raise nodes R1 to R8, Exit)
Edges E	27 (1 from Start, 18 from decisions, 8 into Exit)
V(G) = E - N + 2	27 - 19 + 2 = 10
Decision nodes + 1	9 + 1 = 10 (both methods agree)
Line-level count (line 48 as one decision)	7 decisions, V(G) = 8 (only for comparison)

2.2 Independent basis paths

Path	Route	Representative test
P1	D1 true to R1 (negative dues)	BVA-15
P2	D1 false, D2 false to R2	BVA-19 (WB04)

Path	Route	Representative test
P3	D2 true, D3 false to R2	BVA-18 (WB05)
P4	D3 true, D4 true to R2	ECP-15 (WB06)
P5	D4 false, D5 true to R3 (DUES_BLOCK)	WB01 (added)
P6	D5 false, D6 true to R4 (ALREADY_REGISTERED)	ECP-04
P7	D6 false, D7 true to R5 (PREREQUISITES_MISSING)	ECP-08
P8	D7 false, D8 true to R6 (CREDIT_LIMIT)	ECP-10
P9	D8 false, D9 true to R7 (WAITLISTED)	WB08 (cannot run on the original code)
P10	D9 false to R8 (REGISTERED)	ECP-02 (baseline)

Finding: path P9 cannot be reached in the original code. To make line 58 true we need enrolled > capacity, but line 48 already raises an error for that input, so line 59 is dead code. It can only be reached after the fix of line 58 (defect #5). So the original code had 9 working paths, and the fix brings back the tenth.

2.3 Condition coverage and MC/DC

Atomic conditions in registration_decision(): c1 (line 46) dues < 0; A, B, C (line 48); c5 (line 50) dues above the threshold; c6 (line 52) already registered; c7 (line 54) prerequisites not satisfied; c8 (line 56) credit limit not ok; c9 (line 58) enrolled > capacity. The tests cover each of them as true and as false, except c9 true, which cannot happen on the original code (see above).

Row	A	B	C	Decision	Test
1	F	not evaluated	not evaluated	True (raise)	WB04: enrolled -1, capacity 30
2	T	F	not evaluated	True (raise)	WB05: enrolled 31, capacity 30
3	T	T	T	True (raise)	WB06: enrolled 0, capacity 0
4	T	T	F	False (continue)	WB07: enrolled 0, capacity 30

Python stops evaluating an 'or' as soon as one part is true, so only these four combinations can happen. Independence pairs (masking MC/DC): A is shown by rows 1 and 4 (A changes and the decision changes; B and C are masked in row 1); B by rows 2 and 4 (A stays true, B changes); C by rows 3 and 4 (A and B stay true, C changes). Three conditions need four tests (n + 1), and each condition was shown to change the result on its own. The second compound decision is line 70 of register_course() (decision == WAITLISTED and student_id not in waitlist). It is covered by WB09 (both true), WB10 (second false) and WB11 (first false).

2.4 White-box tests (tests/test_whitebox.py)

Test	Target
WB01	line 50 true branch, line 51, path P5 (dues 2000 gives DUES_BLOCK)
WB02	line 50 decision at the code's constant 1000; FR-04 says any dues > 0 must block
WB04 to WB07	MC/DC rows 1 to 4 of the line 48 decision (A, B, C)
WB08	line 58 true branch, line 59, path P9 (full course gives WAITLISTED)
WB09, WB10, WB11	line 70 compound decision: add to waitlist / already waiting, no duplicate / registered, waitlist not touched

There are 10 white-box tests (WB01, WB02, WB04 to WB11). On the original code 7 pass and 3 fail (WB02, WB08, WB09) because of the defects. After the fixes all of them pass. Running all 66 tests on the original code gives **19 failed, 47 passed** (16 black-box failures + 3 white-box failures); see Figure 9.

3. Statement and branch coverage

Measured with `python -m pytest tests/ --cov=registration --cov-branch --cov-report=term-missing`. pytest-cov gives statement and branch coverage only. It does not measure condition coverage or MC/DC, so we calculated those by hand in section 2.3.

Stage	Statements	Branches	pytest-cov total	Uncovered lines
Initial: black-box tests only, original code	54/57 = 94.7%	23/26 = 88.5%	93%	51, 59, 71
After white-box tests, original code	55/57 = 96.5%	24/26 = 92.3%	95%	59, 71
After defect fixes (PR #8), all 66 tests	58/58 = 100%	26/26 = 100%	100%	none

Gaps and how we handled them. Line 51 (the DUES_BLOCK return) was never reached, because the black-box dues values (500 and 1) are below the code's wrong threshold of 1000. WB01 (dues 2000) reaches it. Lines 59 and 71 (the waitlist return and the waitlist append) cannot be reached on the original code because of defect #5. WB08 and WB09 were written for them, they fail until the fix, and they cover those lines afterwards. Getting 100% was not the goal by itself. The coverage report is what showed us the unreachable path.

4. Defects, GitHub Issues and fix status

Issue	Defect (requirement)	Failing tests	Cause and fix	Status
#2	Dues below Rs. 1001 do not block registration (FR-04)	ECP-01/DT-01, BVA-13, DT-07, DT-11, ST04, WB02	line 50 used '> 1000'; changed to '> 0'	Fixed in PR #8, closed
#3	Prerequisites accepted when only some are completed (FR-03, FR-06)	ECP-07/DT-03, ECP-18	line 41 used intersection (any); changed to required.issubset(completed)	Fixed in PR #8, closed
#4	A total of exactly 18 credits is rejected (FR-01, FR-07)	BVA-03, BVA-05, BVA-07, BVA-23	line 34 used '< 18'; changed to '<= 18'	Fixed in PR #8, closed
#5	Full course is not waitlisted (FR-08, FR-12)	ECP-12/BVA-17/DT-05, BVA-21, ST02, ST03, WB08, WB09	line 58 used 'enrolled > capacity', which is never true after line 48; changed to '=='	Fixed in PR #8, closed
#6	Withdrawal does not decrease course enrollment (FR-13)	ST09	withdraw_course never reduced enrolled; added course.enrolled -= 1	Fixed in PR #8, closed

Sixteen failing black-box tests come from five different defects, so there is one Issue per defect and not one per failing test (ST03 failed only because of defect #5). Each Issue records the requirement ID, the steps to reproduce, the expected and actual output, the failed test IDs and the status. The fixes were made on a separate branch (defect-fixes) after the failing evidence was saved, and PR #8 closed all five Issues.

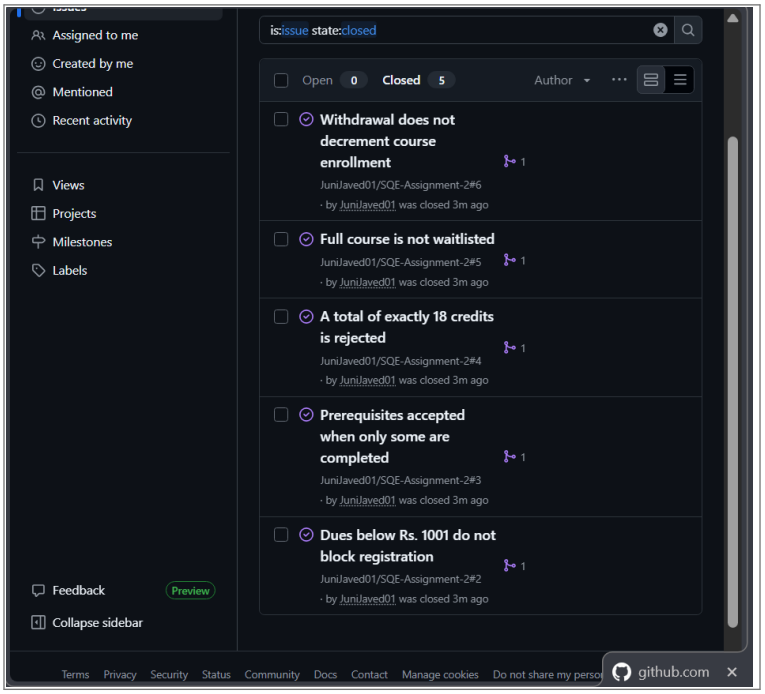


Figure 3. GitHub Issues: Open 0, Closed 5 (Issues #2 to #6), closed by PR #8.

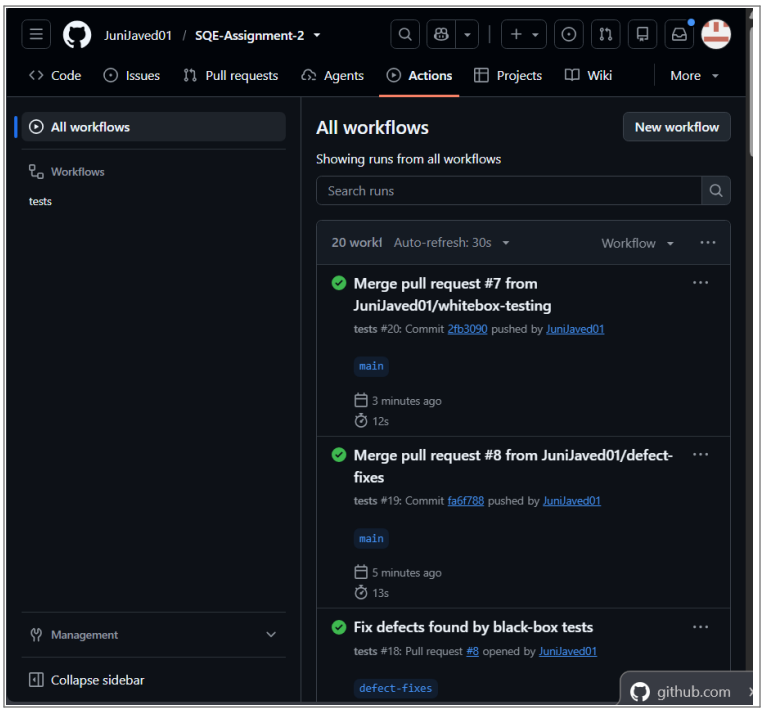


Figure 4. GitHub Actions after the fixes: runs #18 (PR #8), #19 (merge of PR #8) and #20 (merge of PR #7) are green on main.

5. GitHub repository, pull requests and contributions

Repository: https://github.com/JuniJaved01/SQE-Assignment-2. Files: registration.py, requirements.txt, README.md, REQUIREMENTS.md, tests/test_blackbox.py, tests/test_whitebox.py, docs/ (blackbox_test_design.xlsx, decision_table.xlsx, cfg.png, testing_report.pdf) and .github/workflows/tests.yml. The workflow runs pytest with coverage on every push and pull request, and saves the test output and coverage.xml even when tests fail.

5.1 Pull requests and reviews

PR	Branch	Author	Reviewer	Content and outcome
#1	blackbox-testing	Junaid Javed	Mian Ayan	Black-box suite, ECP/BVA/decision-table sheets and the CI workflow. Mian Ayan left review comments (why the expected value is 18, and a clearer test name) and Junaid answered them. CI was red on purpose (16 failed, 40 passed). Merged.
#7	whitebox-testing	Mian Ayan	Junaid Javed	White-box tests (test_whitebox.py): MC/DC, branch and path tests. Junaid reviewed it and noted that WB08 depends on Issue #5. CI was red before the fixes (3 expected failures). Merged.
#8	defect-fixes	Junaid Javed	-	Five defect fixes in registration.py and docs/cfg.png; closes Issues #2 to #6. CI green. Merged.

5.2 Member contributions

Member	Contribution
Junaid Javed (22P-9163)	Requirement analysis and black-box design (ECP, BVA, decision table, optimization); both Excel sheets; tests/test_blackbox.py; repository setup and Actions workflow; Issues #2 to #6; CFG, V(G), MC/DC and coverage analysis; defect fixes (PR #8); PR #1; review of PR #7; this report.
Mian Ayan (22P-9149)	Accepted the collaborator invite and reviewed PR #1 with comments. Created the branch whitebox-testing, added tests/test_whitebox.py (10 white-box tests: WB01, WB02, WB04 to WB11) and opened PR #7. Added a commit to PR #7 so the GitHub Actions run started, then checked the result: 19 failed, 47 passed, with WB02, WB08 and WB09 as the 3 expected failures. The screenshots are in section 5.3.

5.3 Evidence of the white-box work (PR #7)

These screenshots show the steps for the white-box part, from creating the file to the final test result on GitHub Actions.

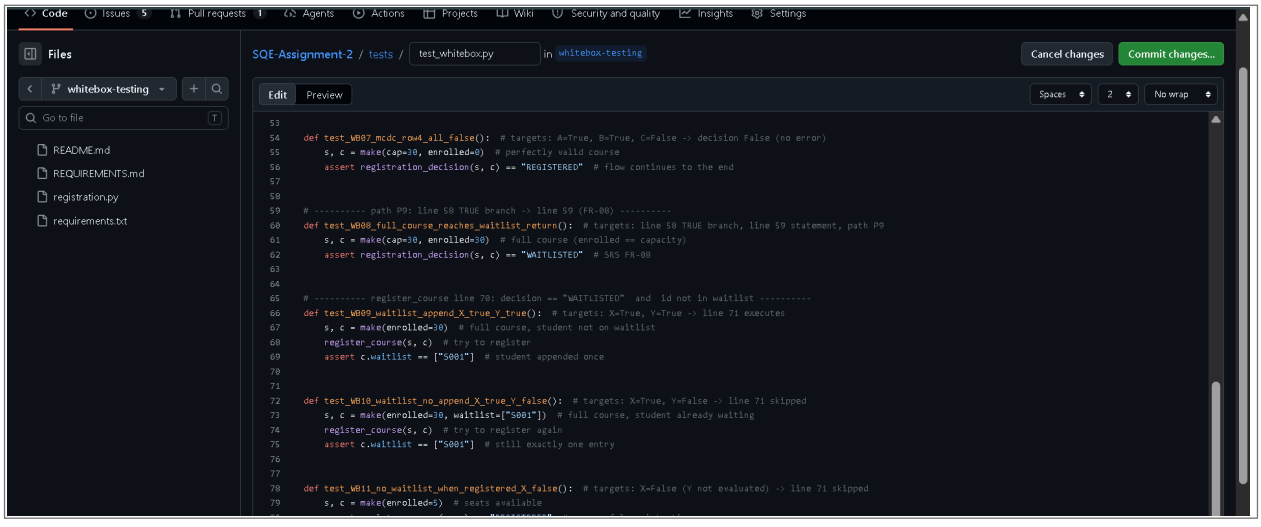


Figure 5. test_whitebox.py being added in the tests folder on the whitebox-testing branch (WB07 to WB11 visible).

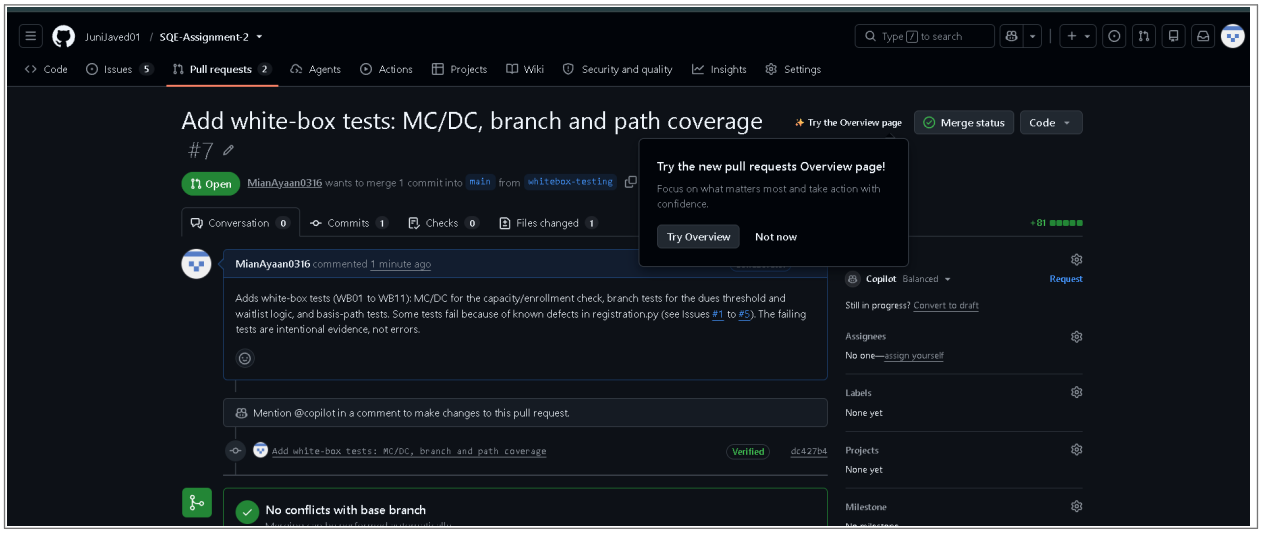


Figure 6. Pull Request #7 "Add white-box tests: MC/DC, branch and path coverage" opened from whitebox-testing into main.

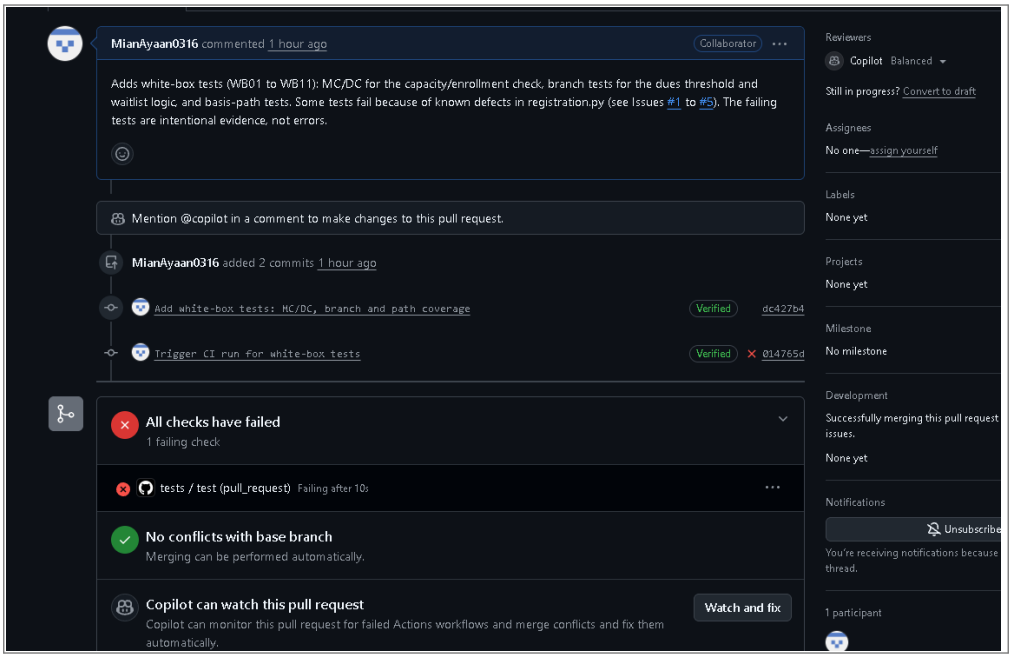


Figure 7. PR #7 with its two commits and the check "tests / test (pull_request)" failing, as expected before the fixes.

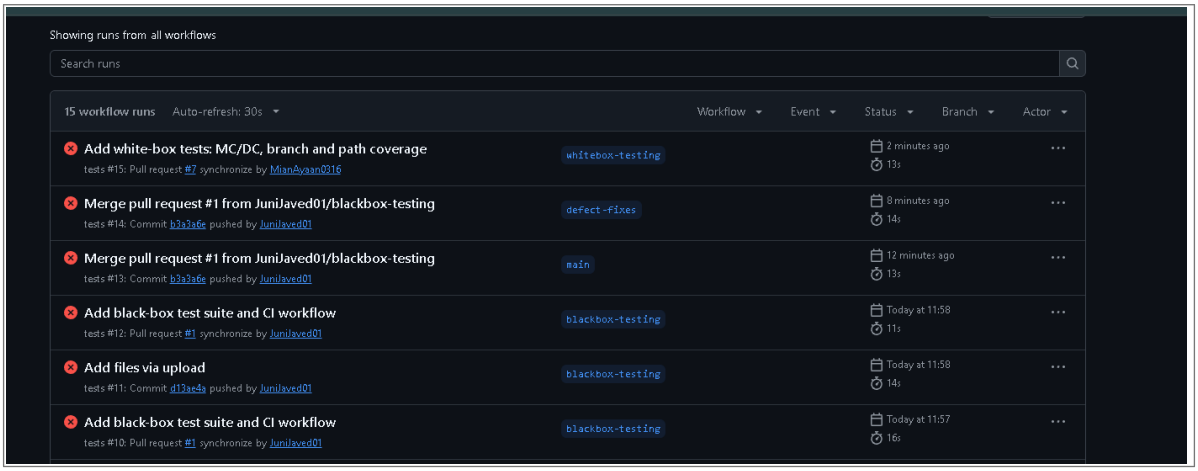


Figure 8. Actions list: run tests #15 (Pull request #7, branch whitebox-testing) next to the earlier black-box runs.

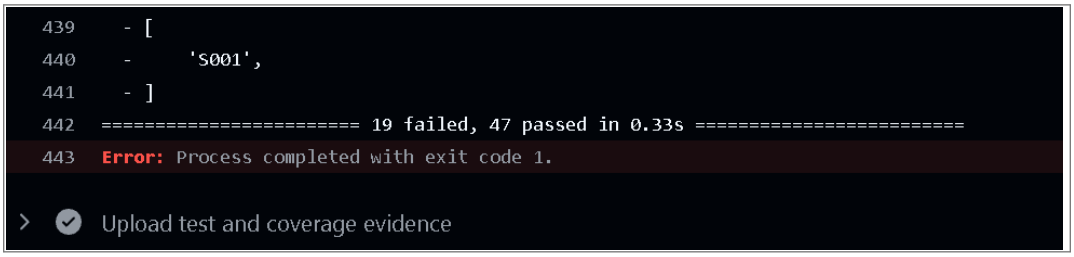


Figure 9. End of the "Run tests with coverage" step for PR #7: 19 failed, 47 passed. The 3 white-box failures are WB02, WB08 and WB09.

6. Conclusion

Tests made from the requirements found five real defects that we could not have seen from the code alone, and we kept the failing runs as evidence before fixing anything. The white-box analysis showed an unreachable path (line 59) caused by defect #5 and gave V(G) = 10. After the five one-line fixes, all 66 tests pass with 100% statement and branch coverage, and the GitHub Actions run on main is green. No expected value was changed to make a test pass.